ATIVIDADE AVALIATIVA - DESAFIO DEVOPS

Cidades ESG Inteligentes

Pipeline CI/CD, teste automatizados, containers e orquestração

INTEGRANTE	RM
ERICLES MOURA	**564492**
KLEBER WILLIAM	561315
JULLIA RODRIGUES	565591
JUAN	PREENCHER RM

1. Resumo do projeto

O problema, a solução e o valor do fluxo DevOps

O projeto é uma API REST que registra indicadores ambientais de cidades e calcula um ranking ESG. A aplicação combina quatro indicadores: energia renovavel 35%, reciclagem 25%, área verde por habitante normalizada 20% e qualidade do transporte público 20%. O resultado é uma nota de 0 a 100.

Analogia simples: a API é como uma ficha escolar das cidades. Cada indicador é uma materia, cada materia tem um peso, e o ranking ordena as melhores medias.

Camada	Responsabilidade	Tecnologia
API	Receber, validar e responder requisições HTTP	Spring Web + Bean Validation
Negócio	Calcular score e ordenar o ranking	Java 21
Dados	Persistir cidades e versionar o schema	JPA/Hibernate + Flyway
Banco	Armazenar dados nos ambientes Docker	PostgreSQL 17

Operação	Expor saúde, prontidão e métricas	Spring Actuator
Entrega	Construir, testar e promover o artefato	GitHub Actions + Docker

Arquitetura lógica

CLIENTE HTTP	API SPRING BOOT	POSTGRESQL
GET/POST/PUT/DELETE	REST + regras + health	dados + volume

O Docker Compose cria uma rede isolada para a comunicação entre API e banco, um volume para não perder os dados e variáveis de ambiente para separar configuração de código.

2. Aplicação e recursos

Endpoints, validação e regra de pontuação

Método	Rota	Objetivo
GET	/api/cities	Listar cidades
GET	/api/cities/ranking	Ordenar cidades pelo score ESG
GET	/api/cities/{id}	Consultar uma cidade
POST	/api/cities	Cadastrar com validação
PUT	/api/cities/{id}	Atualizar indicadores
DELETE	/api/cities/{id}	Remover cadastro
GET	/actuator/health	Verificar saúde do serviço

Regra de negócio

```
score = energia_renovavel x 0,35
+ reciclagem       x 0,25
      + area_verde_norm  x 0,20
      + transporte       x 0,20
```

A área verde é limitada a 100 pontos após a normalização. O resultado final é arredondado para duas casas decimais. Os dados inválidos retornam HTTP 400 no formato Problem Details; uma cidade inexistente retorna HTTP 404.

Qualidade incorporada

Recurso	Como reduz risco
Bean Validation	Impede percentuais fora de 0 a 100, UF inválida e população menor que 1
Tratamento global	Padroniza erros para quem consome a API
Migração Flyway	Recria o mesmo schema de forma versionada

Health/readiness	Permite ao orquestrador detectar se o serviço está pronto
Dados iniciais	Facilita demonstração em ambiente local e staging

3. Pipeline CI/CD

Automação de build, testes e deploy em dois ambientes



Etapa	Lógica e critério
1. Build e testes	Java 21 + Maven executam clean verify. Falha interrompe a esteira.
2. Imagem	Docker constrói um artefato único e o publica para os jobs seguintes.
3. Staging	Compose sobe API e PostgreSQL na porta 8081; health e ranking são testados.
4. Produção	A mesma imagem passa para a porta 8082 somente depois de staging.
5. Evidências	JUnit, logs e JSONs de health são anexados ao run do GitHub Actions.

Analogia: o pipeline funciona como uma linha de inspeção. O produto só muda de estação quando passa pela verificação anterior; a caixa enviada a staging é exatamente a mesma enviada à produção.

O arquivo .github/workflows/ci-cd.yml é disparado em pull requests, pushes para main e manualmente. Os ambientes do GitHub registram staging e production; production pode receber aprovação obrigatória em Settings > Environments.

4. Containerização e orquestração

Como a aplicação vira um pacote repetível

Dockerfile multiestágio

```
FROM maven:3.9.11-eclipse-temurin-21-alpine AS build
WORKDIR /workspace
COPY pom.xml .
RUN --mount=type=cache,target=/root/.m2 mvn -B dependency:go-offline
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -B clean package

FROM eclipse-temurin:21-jre-alpine AS runtime
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring
COPY --from=build --chown=spring:spring /workspace/target/*.jar app.jar
USER spring:spring
EXPOSE 8080
HEALTHCHECK CMD wget -qO- http://localhost:8080/actuator/health/readiness || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "/app/app.jar"]
```

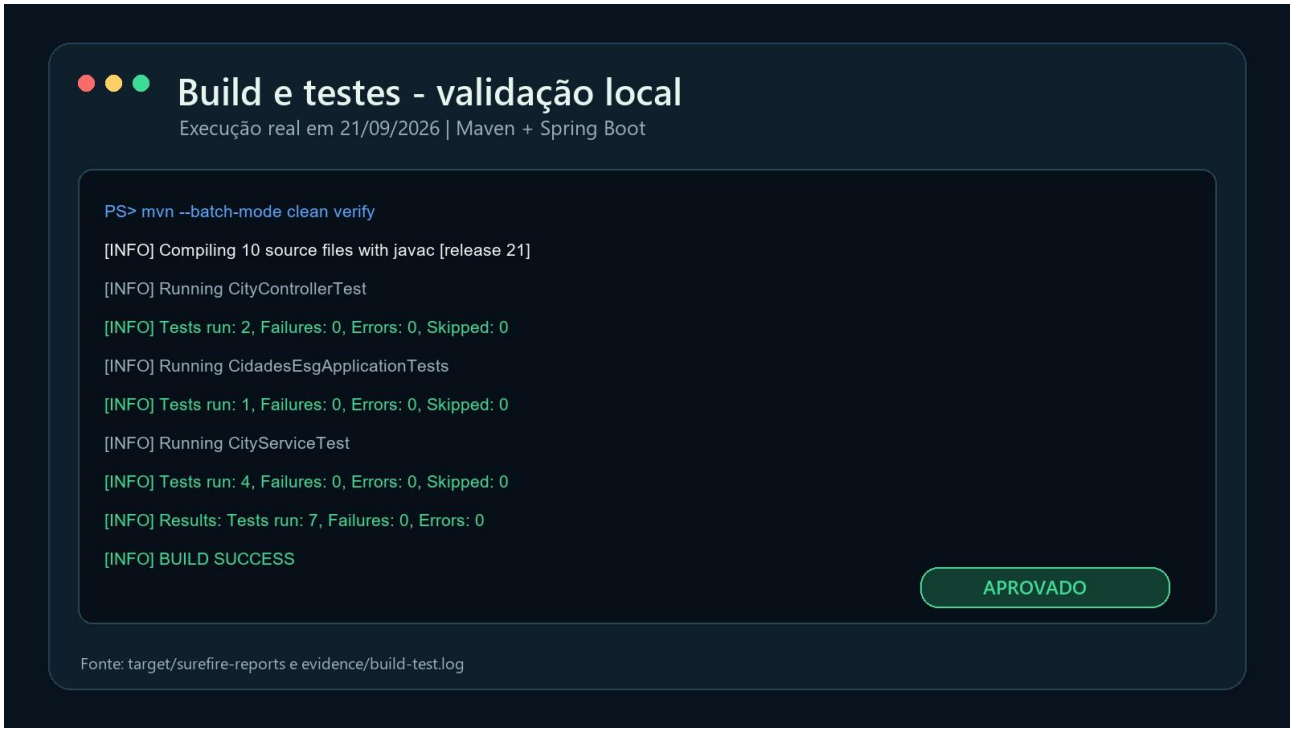
Decisão	Benefício
Multiestágio	Maven e arquivos de build não entram no final
Cache Maven	Builds seguintes baixam menos dependências
Usuário spring	Reduz o impacto de uma eventual invasão
Health check	Docker sabe distinguir processo aberto de aplicação pronta
Limite de RAM	JVM se adapta à memória concedida ao container

Docker Compose

Elemento	Implementação
Serviços	app (Spring Boot) e database (PostgreSQL 17)
Volume	postgres-data em /var/lib/postgresql/data
Rede	esg-network do tipo bridge
Variáveis	DB_URL, DB_USER, DB_PASSWORD, perfil, porta e seed
Dependência	app aguarda o health check do PostgreSQL
Overrides	arquivos separados para staging e production

5. Evidência de build e testes

Resultados reais obtidos nesta máquina



Suíte	Testes	Resultado
CityControllerTest	2	Aprovados
CidadesEsgApplicationTests	1	Aprovado
CityServiceTest	4	Aprovados
Total	7	0 falhas / 0 erros / 0 ignorados

O que foi testado

Inicialização do Spring, cálculo ponderado, ranking decrescente, normalização da UF, erro de cidade inexistente, listagem HTTP e validação de payload. O comando terminou com BUILD SUCCESS e produziu o JAR executável.

6. Evidência da aplicação

Health check e endpoint funcional executados no JAR



O artefato foi compilado em memória h2 para permitir a validação sem Docker. A API respondeu HTTP 200, o Actuator retornou status UP e o ranking trouxe três cidades.

Posição	Cidade	UF	Score ESG
1	Curitiba	PR	77,50
2	Florianópolis	SC	74,25
3	São Paulo	SP	51,07

7. Ambientes de staging e produção

Promoção, isolamento e evidências do GitHub

Característica	Staging	Produção
GitHub Environment	staging	production
Porta do runner	8081	8082
Perfil Spring	staging	production
Dados de exemplo	Ativados	Desativados
Teste do deploy	health + ranking	health
Dependência	build-and-test	deploy-staging

Imagem	cidades-esg:{commit}	a mesma imagem aprovada

Evidências produzidas noo primeiro run

Artefato	Conteúdo
relatorios-de-testes	XML e TXT do JUnit/Surefire
evidencias-staging	deploy.log, health.json e ranking.json
evidencias-production	deploy.log e health.json
Deployments	Histórico visual de staging e production

8. Desafios e soluções

Decisões que aumentam robustez e segurança

Desafio	Solução aplicada
Ambientes diferentes	Promover a mesma imagem imutável para staging e produção
Banco ainda indisponível	Health check do PostgreSQL e depends_on por condição
Senha exposta	.env ignorado; somente .env.example sem credencial real
Deploy que apenas inicia	Smoke test verifica readiness e endpoint funcional
Alteração de schema	Migrações Flyway versionadas no código
Container privilegiado	Usuário spring sem privilégios na imagem final
Falha silenciosa	Qualquer comando com erro encerra o job e impede promoção
Sem Docker local	Perfil H2 demo validou o JAR sem fingir o deploy Docker

Boas práticas recomendadas ao publicar

1	Configurar aprovação obrigatória no environment production.
2	Usar GitHub Secrets para credenciais de um servidor permanente.
3	Não versionar .env, senhas ou tokens.
4	Guardar os artefatos de evidência do run entregue.
5	Criar tag da versão final, por exemplo v1.0.0.

Analogia: Docker é a caixa lacrada; Compose é o mapa que diz onde cada caixa fica; CI/CD é a esteira que inspeciona e leva a caixa de um setor ao outro.

9. Checklist de entrega

Situação atual e últimos passos

Item	Situação
Projeto compactado em ZIP com estrutura organizada	OK
Dockerfile multiestágio	OK
Docker Compose com volume, variáveis e rede	OK
Pipeline com build, testes, staging e produção	OK
README com instruções e evidências locais	OK
Documentação técnica em PDF	OK
Build local e 7 testes aprovados	OK
JAR iniciado, health UP e ranking verificado	OK
Nome e RM preenchidos	OK
Workflow executado no GitHub	PENDENTE
Prints reais de Actions e Deployments	PENDENTE